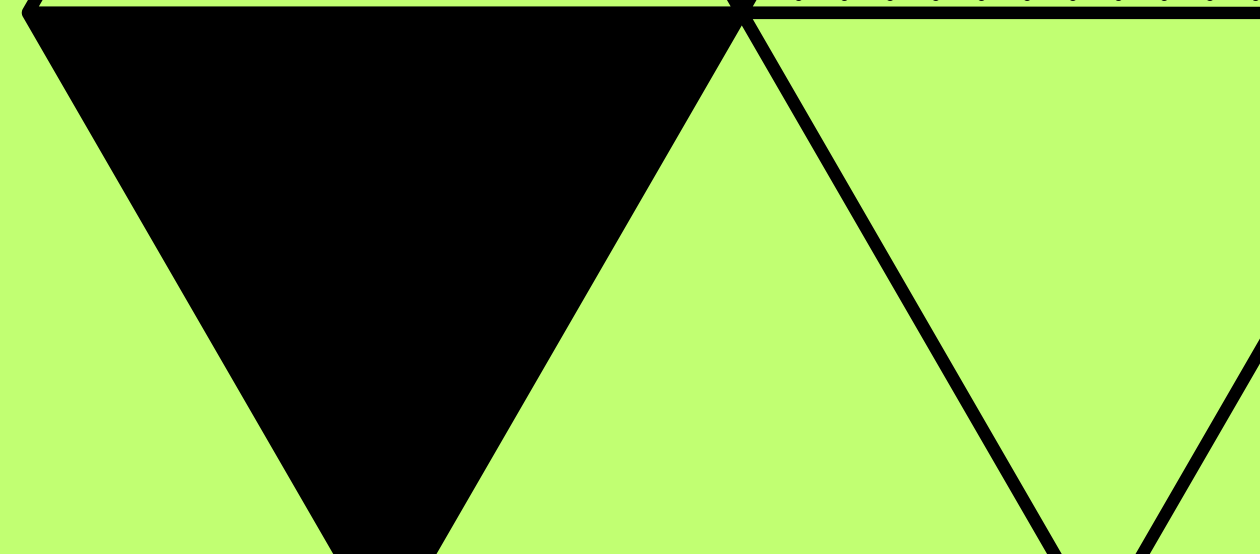
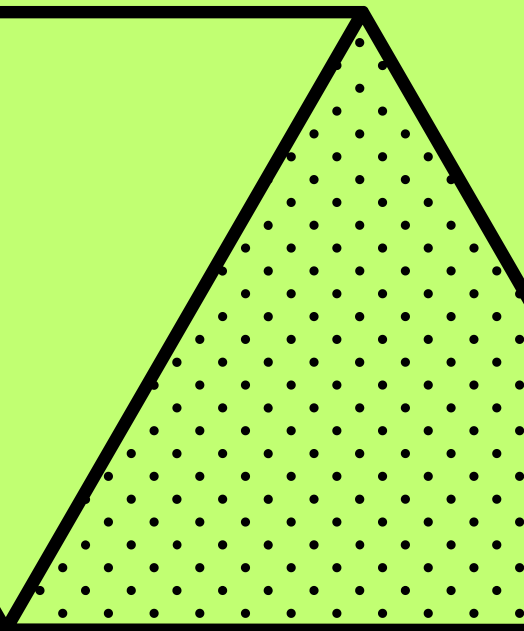
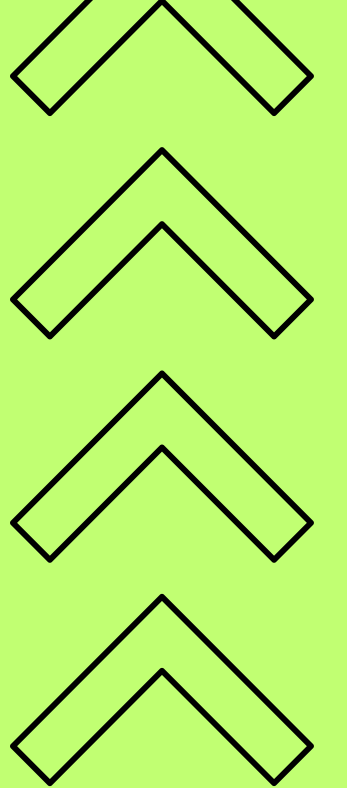
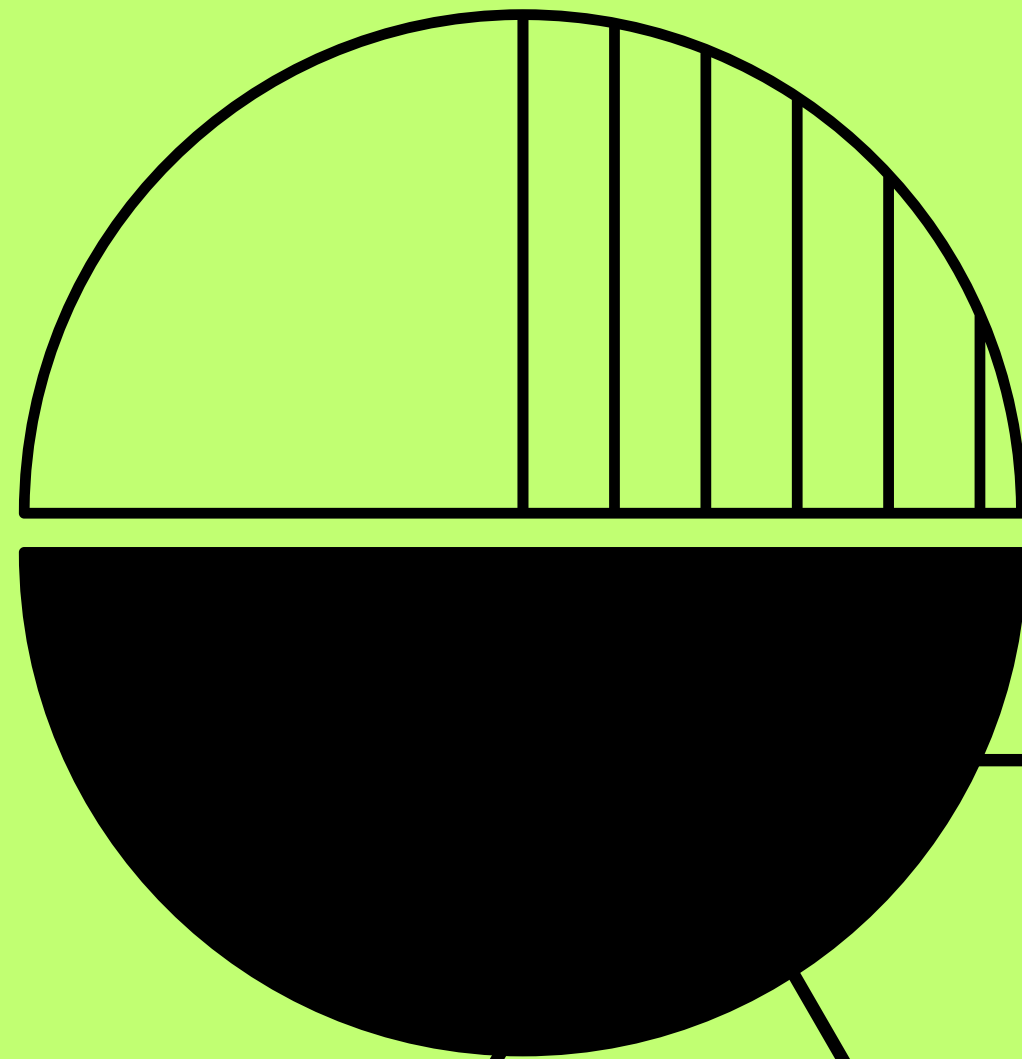
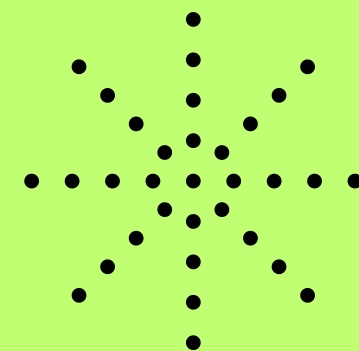


ML project

Comparación de modelos
supervisados para
Social media addiction

Por: Oscar Fernández
14 de mayo de 2026



Introducción

Este dataframe viene de:

Social Media Addiction SMA10 Dataset

Published: 6 January 2025, Version 1

DOI: 10.17632/9mxm455dfm.1

Contributors:

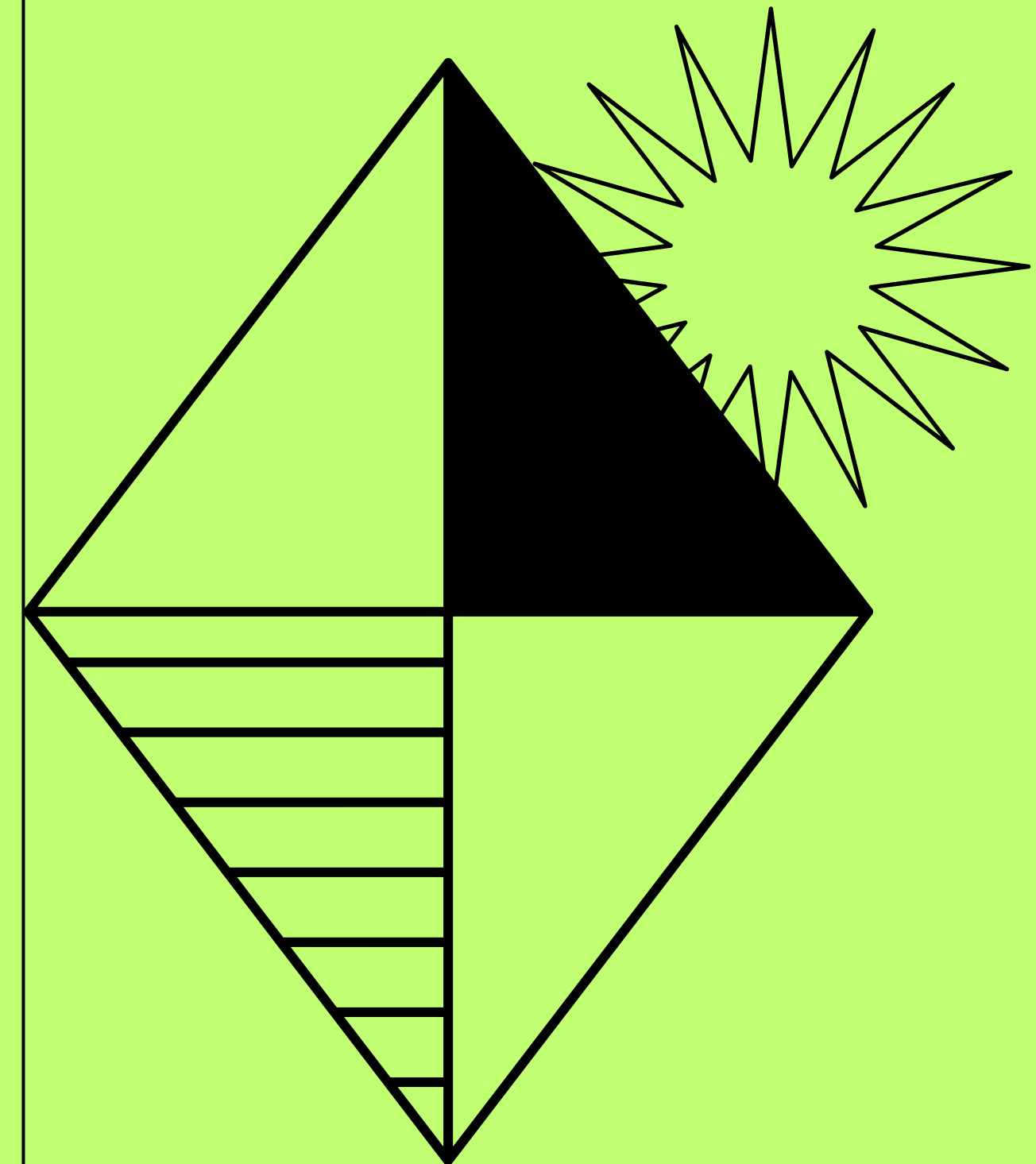
Md. Mukhlesur Rahman, Golam Rabbany, Biplob7991

Biplob Hossain, ABM Alauddin Chowdhury

<https://data.mendeley.com/datasets/9mxm455dfm/1>

Diseñado para medir adicción a las redes sociales en estudiantes universitarios.

La muestra proviene de Daffodil International University (DIU, Bangladesh) e incluye 1.029 participantes.



Cuestionario de Adicción a Redes Sociales (SMAQ, 10 ítems)

Instrucciones: Responde cada pregunta pensando en tu comportamiento durante el último año, usando la escala: (1) Nunca, (2) Rara vez, (3) A veces, (4) A menudo, (5) Siempre.

Q1. Preocupación: ¿Pasa mucho tiempo pensando en las redes sociales o planificando cuándo podrá usarlas?

Q2. Tolerancia: ¿Siente la necesidad de usar las redes sociales cada vez más tiempo para obtener la misma satisfacción?

Q3. Evasión: ¿Utiliza las redes sociales para olvidarse de sus problemas personales o reducir sentimientos de culpa o ansiedad?

Q4. Abstinencia: ¿Se siente inquieto, irritable o ansioso si se le prohíbe usar las redes sociales?

Q5. Conflicto: ¿Ha descuidado sus estudios, trabajo o tareas del hogar por dedicarle tiempo a las redes?

Q6. Pérdida de control: ¿Ha intentado reducir el tiempo que pasa en redes sociales sin éxito?

Q7. Recaída: ¿Suele conectarse por más tiempo del que inicialmente tenía planeado?

Q8. Engaño: ¿Ha mentido a sus familiares o amigos sobre la cantidad de tiempo que pasa conectado?

Q9. Problemas sociales: ¿Ha preferido quedarse en redes sociales en lugar de salir o interactuar con personas físicamente?

Q10. Consecuencias: ¿Sigue usando las redes sociales a pesar de saber que le están causando problemas de sueño, salud o rendimiento?

Abriendo el dataset

El dataset contiene 10 ítems principales (SMAQ1–SMAQ10) que evalúan adicción a redes sociales mediante una escala Likert de 5 puntos: 1 — Nunca, 2 — Rara vez, 3 — A veces, 4 — A menudo y 5 — Siempre

Variables Demográficas:

Q1_Gender: 1 = Hombre, 2 = Mujer

Q2_Living_Area: 1 = Urbano, 2 = Rural

Q3_Marital_Status: 1 = Casado, 2 = Soltero

```
1 mobile10 = pd.read_csv("SMA.csv")
2 mobile10.head()
```

[4]

...

	Q1_Gender	Q2_Living_Area	Q3_Marital_Status	SMAQ1	SMAQ2	SMAQ3	SMAQ4	SMAQ5	SMAQ6	SMAQ7	SMAQ8	SMAQ9
0	1	1	2	4	5	1	3	5	2	2	4	5
1	1	1	2	3	2	1	5	5	1	1	2	5
2	2	1	2	3	2	1	1	1	1	1	1	1
3	1	2	2	2	5	5	2	3	1	3	4	4
4	1	1	2	1	4	3	1	2	1	1	1	1

Explorando y limpiando

Veremos si existen datos NAN, outliers

```
1 mobile10.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 1029 entries, 0 to 1028
Data columns (total 15 columns):
#   Column                Non-Null Count  Dtype
---  -
0   Q1_Gender              1029 non-null  int64
1   Q2_Living_Area         1029 non-null  int64
2   Q3_Marital_Status     1029 non-null  int64
3   SMAQ1                 1029 non-null  int64
4   SMAQ2                 1029 non-null  int64
5   SMAQ3                 1029 non-null  int64
6   SMAQ4                 1029 non-null  int64
7   SMAQ5                 1029 non-null  int64
8   SMAQ6                 1029 non-null  int64
9   SMAQ7                 1029 non-null  int64
10  SMAQ8                 1029 non-null  int64
11  SMAQ9                 1029 non-null  int64
12  SMAQ10                1029 non-null  int64
13  SMA_Scale_value       1029 non-null  int64
14  SMA_Scale (Class_Label) 1029 non-null  int64
dtypes: int64(15)
memory usage: 120.7 KB
```

```
1 Q1 = df.quantile(0.25)
2 Q3 = df.quantile(0.75)
3 IQR = Q3 - Q1
4
5 outliers_iqr = ((df < (Q1 - 1.5 * IQR)) | (df > (Q3 + 1.5 * IQR)))
6
7 print(outliers_iqr.sum())
```

```
1 from sklearn.ensemble import IsolationForest
2
3 iso = IsolationForest(contamination=0.05)
4 df["outlier"] = iso.fit_predict(X)
5
6 print(df["outlier"].value_counts())      ## 52 outliers!! (-1 significa outlier)
```

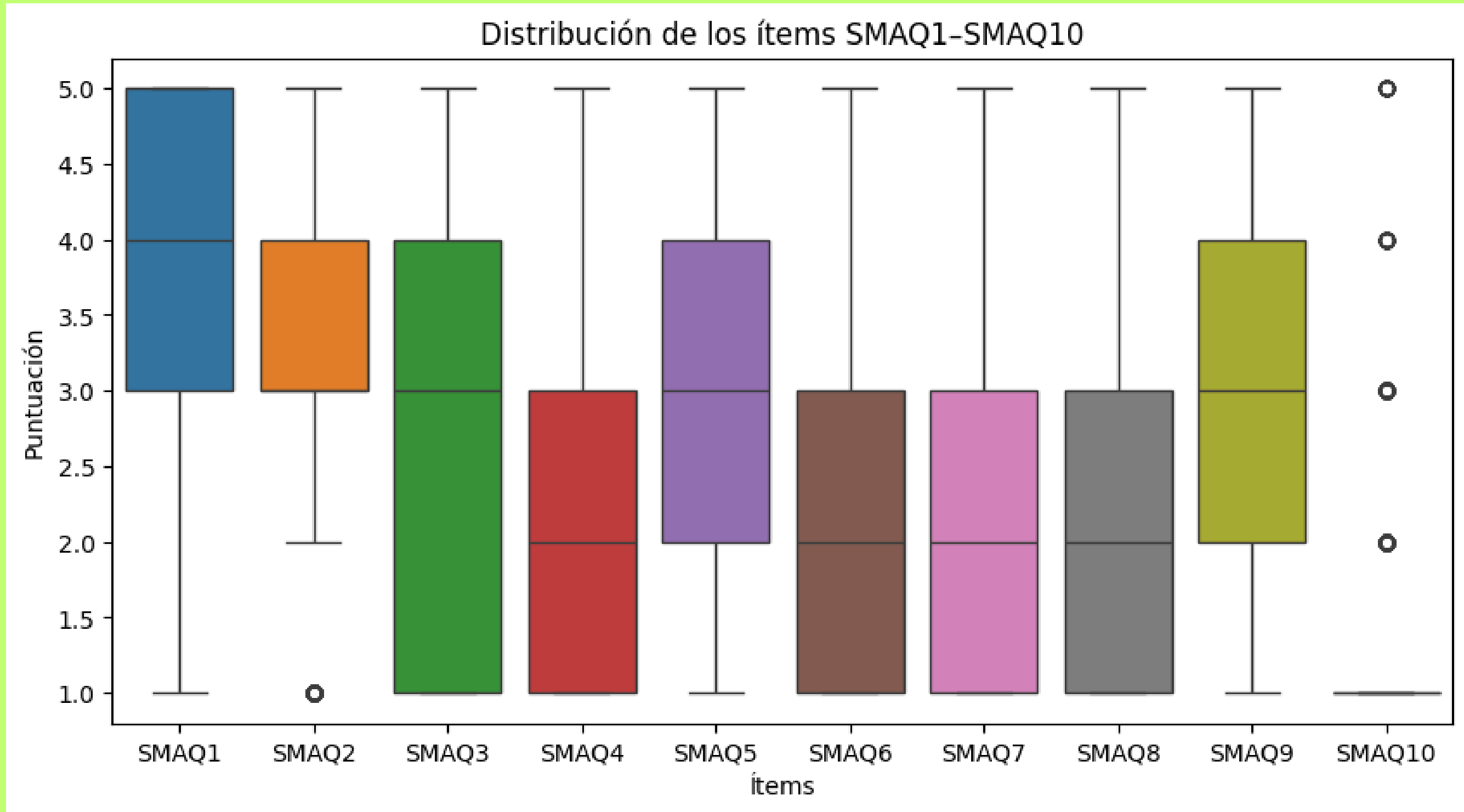
```
outlier
1      977
-1      52
Name: count, dtype: int64
```

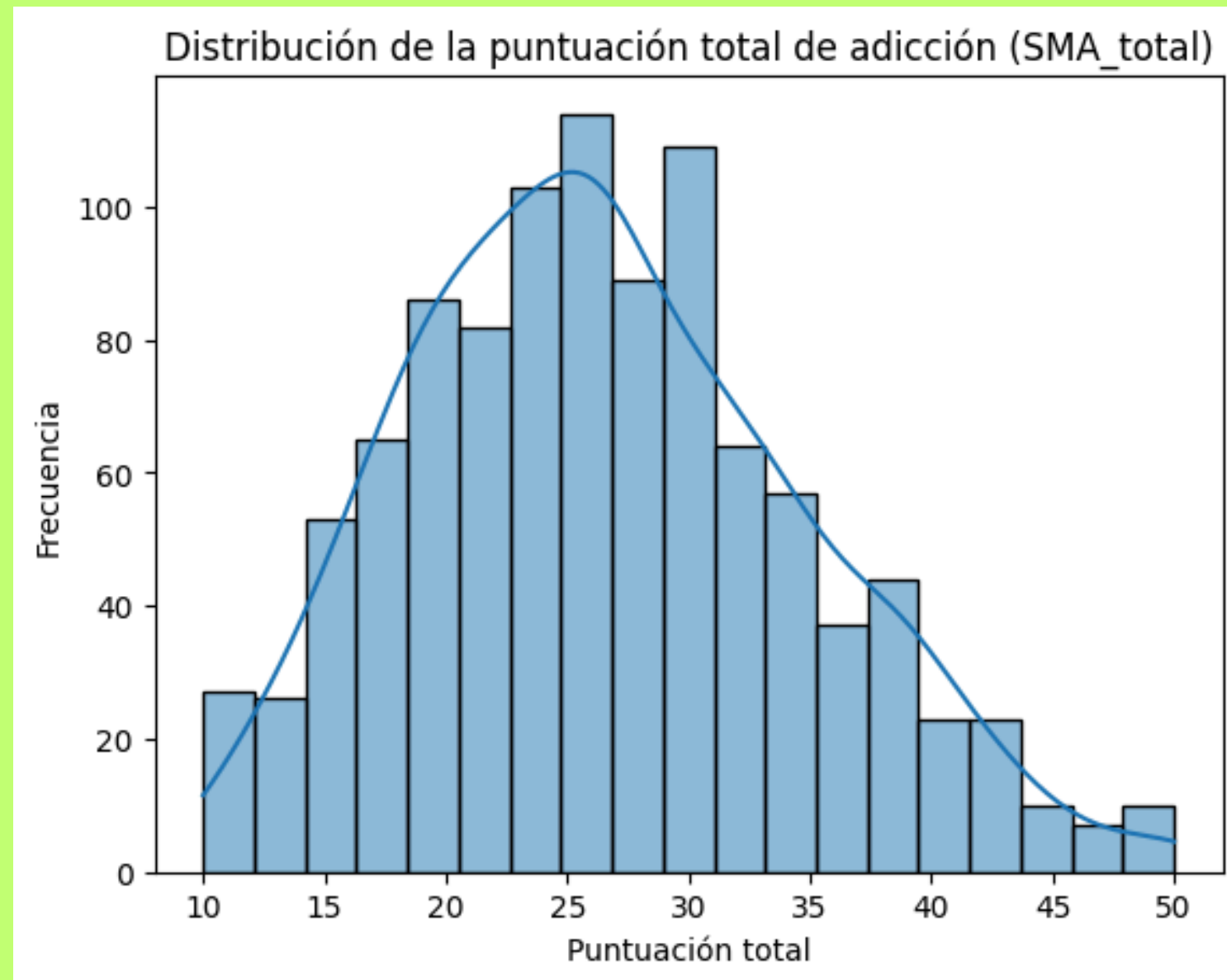
Estos outliers deben conservarse porque representan puntos de adicción importante

Descripción de los ítems SMAQ:

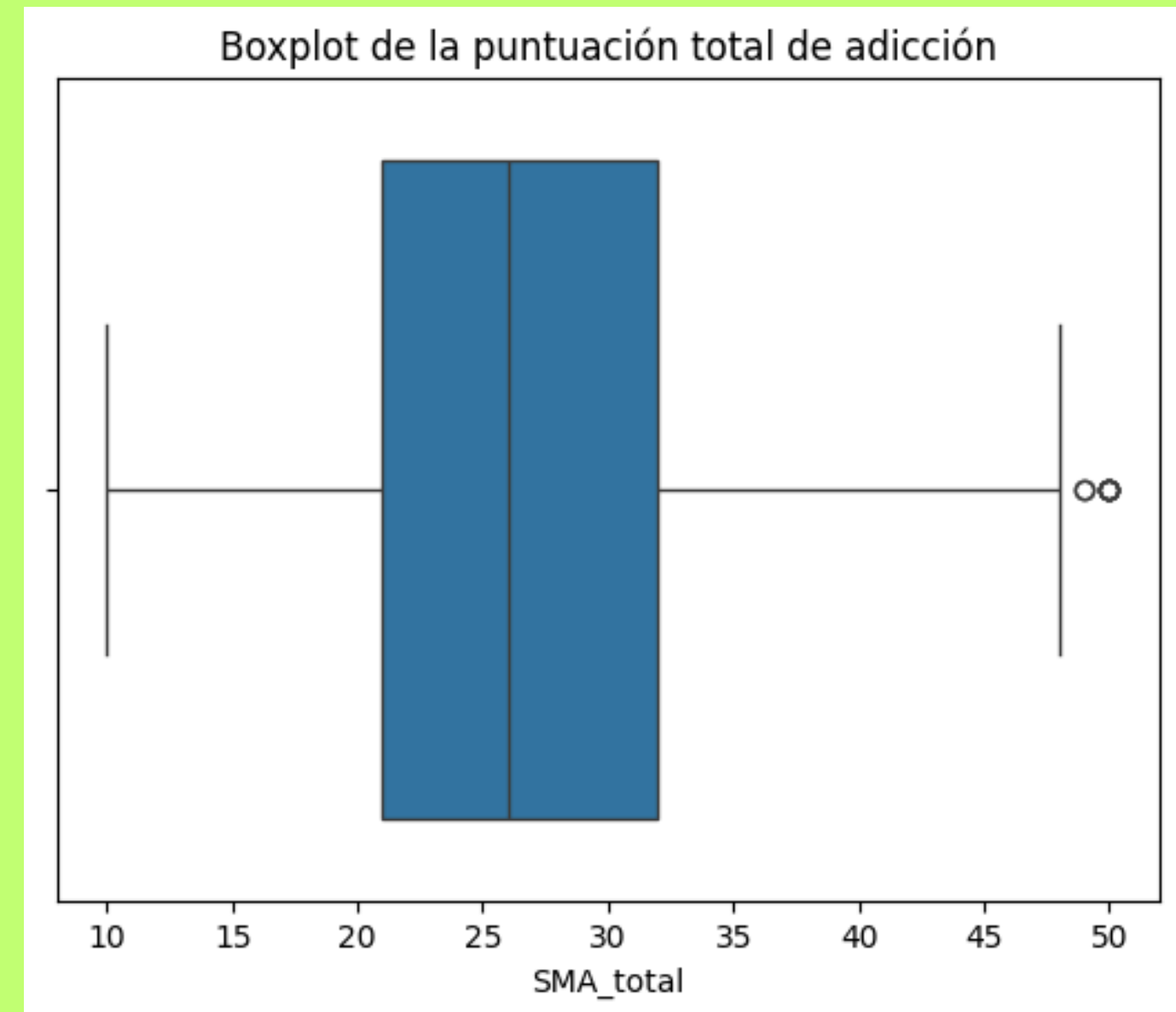
	SMAQ1	SMAQ2	SMAQ3	SMAQ4	SMAQ5 \
count	1029.000000	1029.000000	1029.000000	1029.000000	1029.000000
mean	3.926142	3.335277	2.743440	2.182702	2.898931
std	1.155425	1.262035	1.380192	1.239607	1.412661
min	1.000000	1.000000	1.000000	1.000000	1.000000
25%	3.000000	3.000000	1.000000	1.000000	2.000000
50%	4.000000	3.000000	3.000000	2.000000	3.000000
75%	5.000000	4.000000	4.000000	3.000000	4.000000
max	5.000000	5.000000	5.000000	5.000000	5.000000

	SMAQ6	SMAQ7	SMAQ8	SMAQ9	SMAQ10
count	1029.000000	1029.000000	1029.000000	1029.000000	1029.000000
mean	2.389699	2.029155	2.491740	3.204082	1.396501
std	1.373998	1.191378	1.379103	1.364910	0.866714
min	1.000000	1.000000	1.000000	1.000000	1.000000
25%	1.000000	1.000000	1.000000	2.000000	1.000000
50%	2.000000	2.000000	2.000000	3.000000	1.000000
75%	3.000000	3.000000	3.000000	4.000000	1.000000
max	5.000000	5.000000	5.000000	5.000000	5.000000

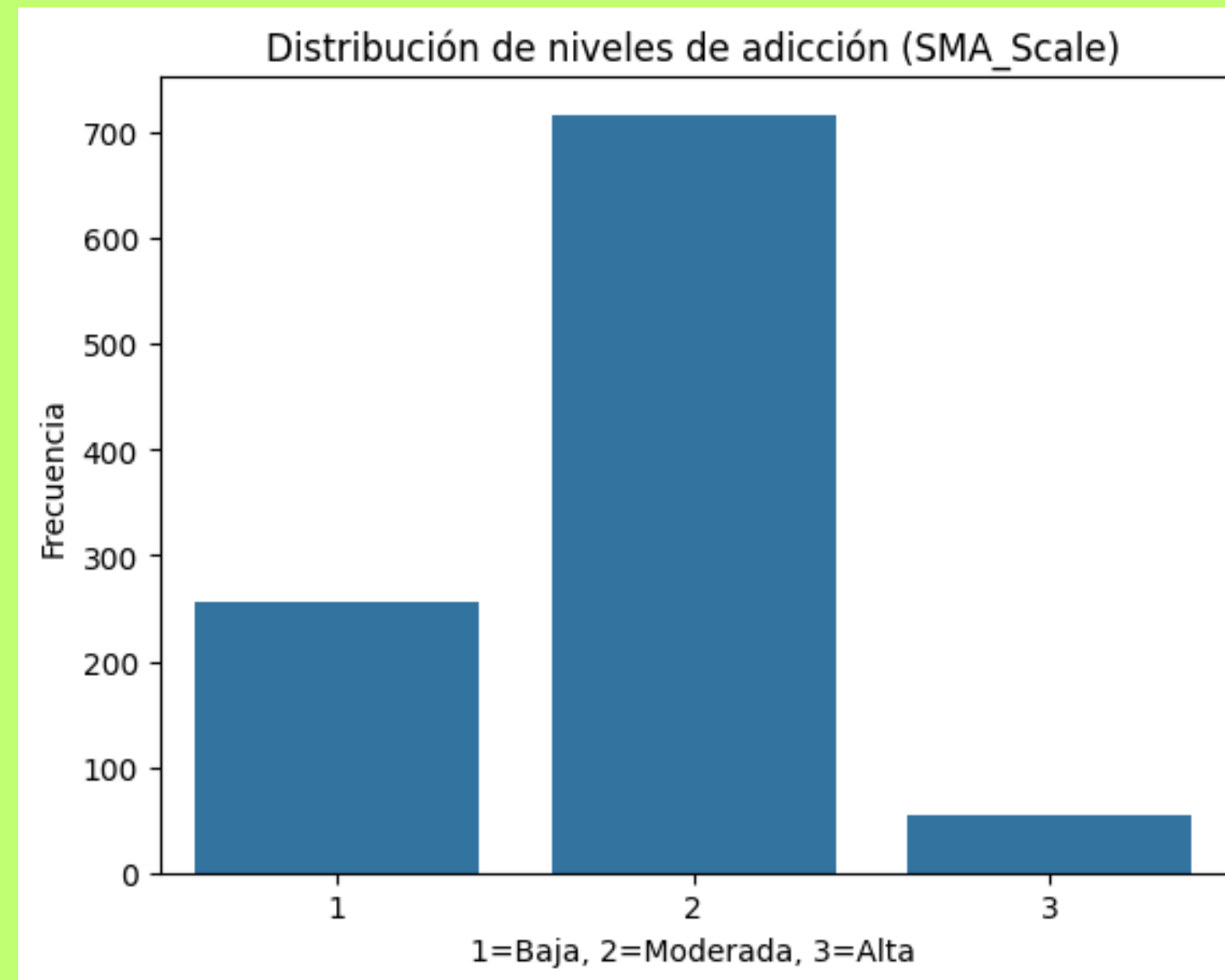




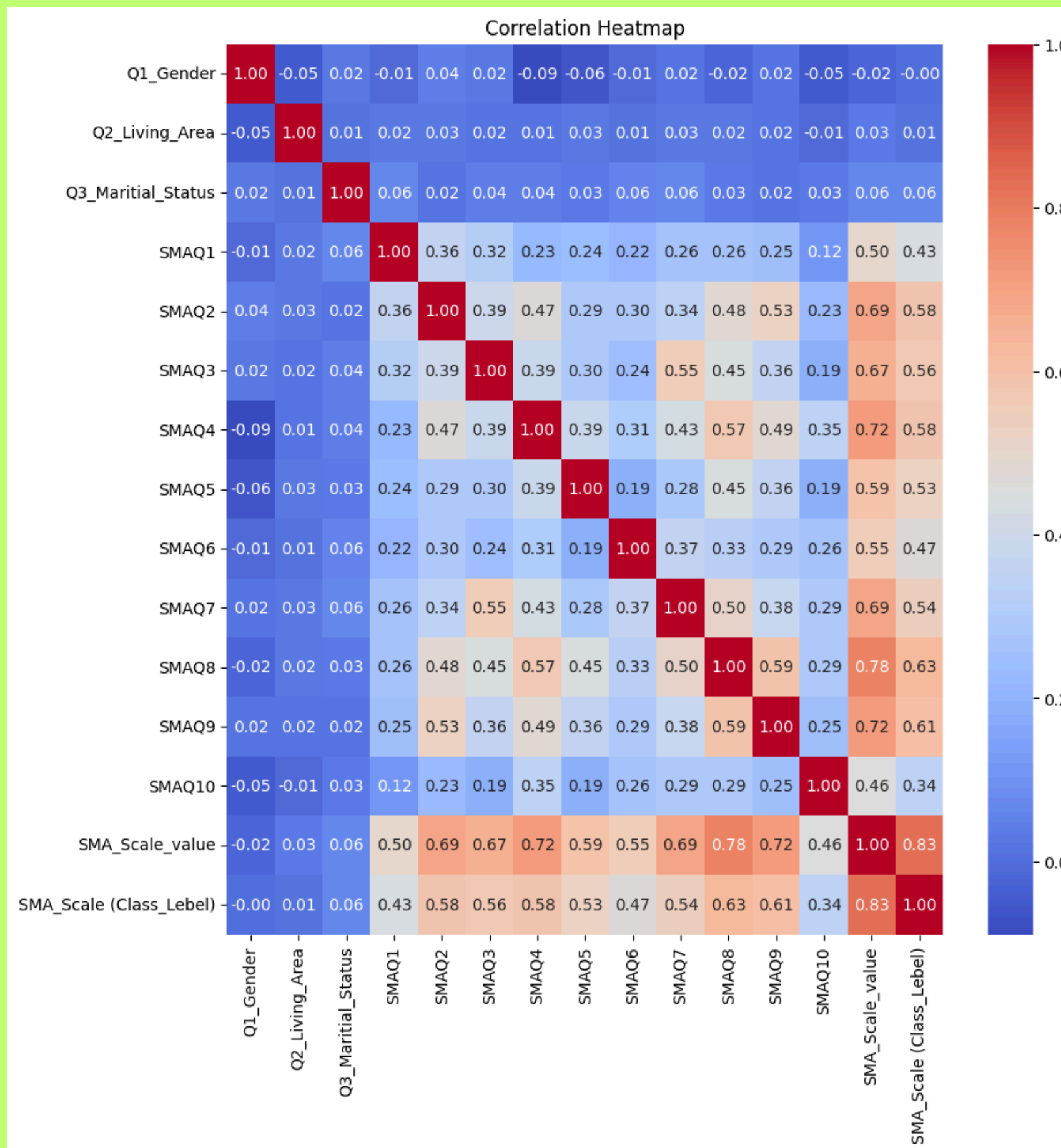
La mayoría puntúa valores medios,
unos pocos valores altos o muy altos (outliers)



Rango amplio de puntuación
y presencia de outliers



La mayoría no tiene adicción, sólo unos 50 casos de 1029



Items muy correlacionados:

Q2 tolerancia - Q9 prob. sociales (0.53)

Q4 abstinencia - Q8 engaño (0.57)

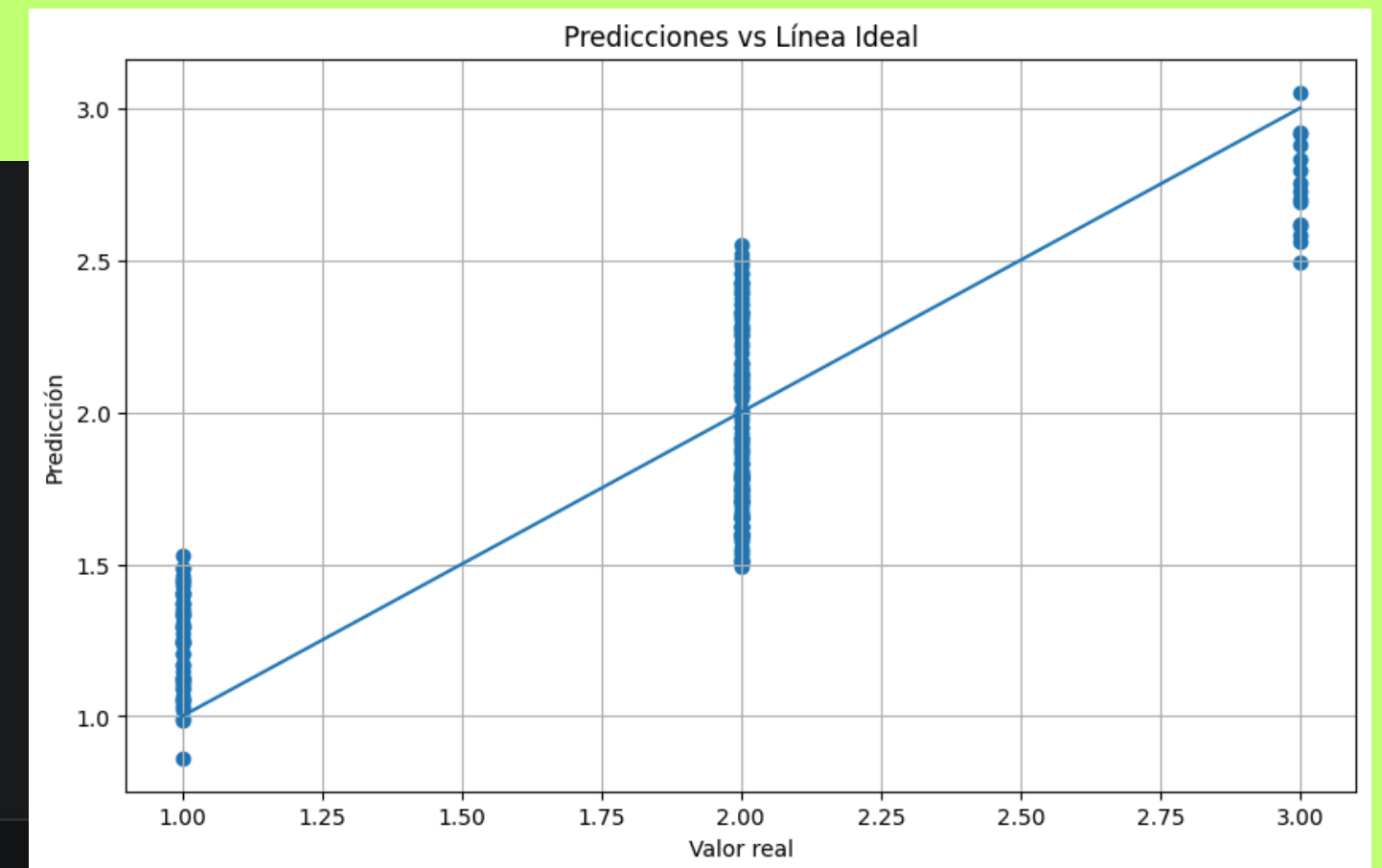
Q8 engaño - Q9 prob. sociales (0.59)

Regresión lineal

Se aplicó regresión lineal como modelo base, aunque no es adecuada para problemas de clasificación. Los resultados obtenidos no son óptimos, lo que refuerza la necesidad de utilizar modelos específicos de clasificación. La regresión lineal daría valores como: 1.3, 2.7, 1.8... tendríamos que redondear.
Problema: No respeta la naturaleza de clases, Resultados poco fiables, Interpretación incorrecta

```
4 model = LinearRegression()
5 model.fit(X_train, y_train)
6
7 y_pred = model.predict(X_test)
8
9 print("MSE:", mean_squared_error(y_test, y_pred))
```

MSE: 0.088615568805453

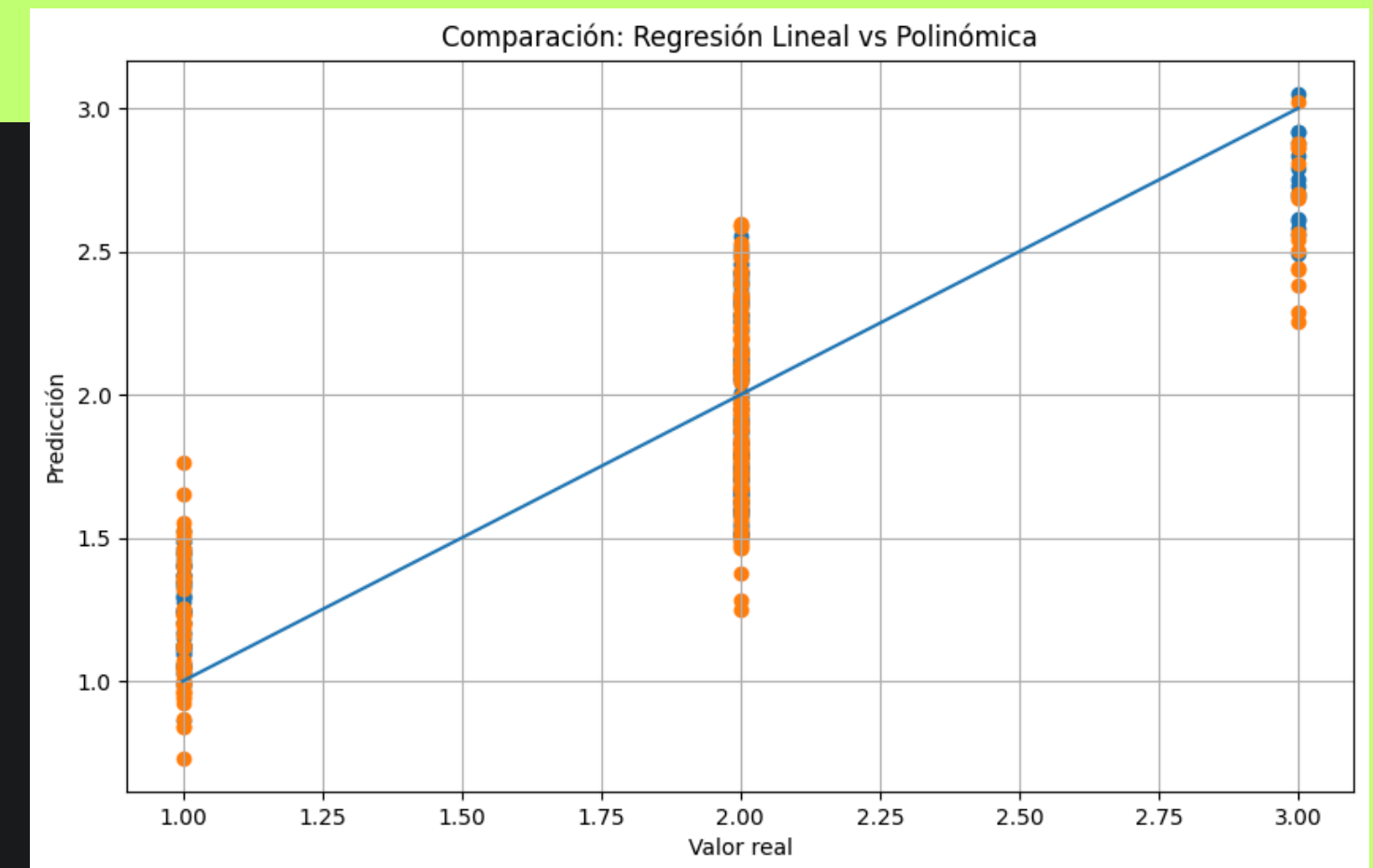


Regresión polinómica

Se aplicó regresión polinómica para capturar relaciones no lineales y comparar su rendimiento con otros modelos. No es adecuada para un problema de clasificación (clases 1, 2 y 3). La regresión predice valores continuos, no respetando las clases. Es difícil de interpretar y puede sobreajustar fácilmente.

```
poly_model = Pipeline([
    ("poly", PolynomialFeatures(degree=2)),
    ("linear", LinearRegression())
])

poly_model.fit(X_train, y_train)
y_pred_poly = poly_model.predict(X_test)
```



MSE: 0.088615568805453

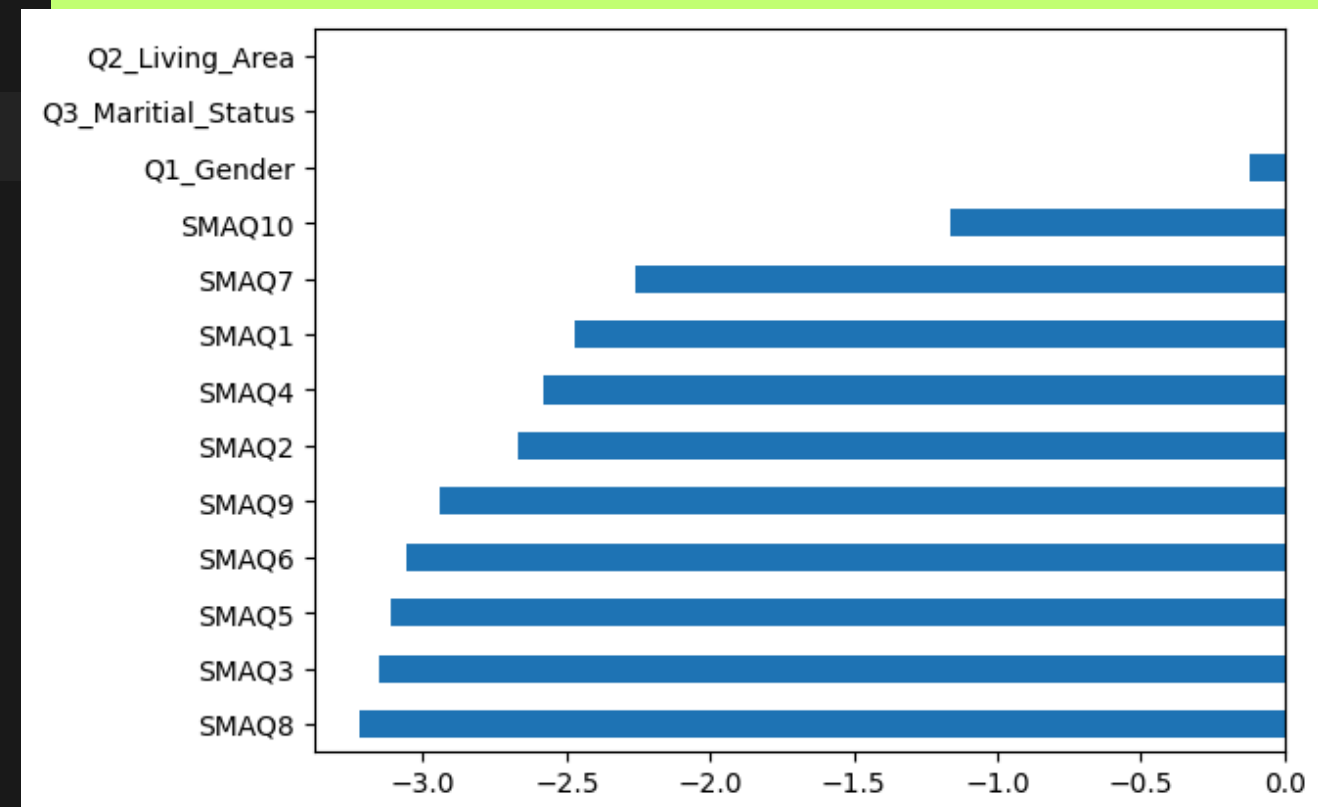
Regresión logística (clasificación)

Se ha aplicado regularización mediante regresión logística con penalización L1 y L2 con el objetivo de reducir el sobreajuste y analizar la relevancia de las variables. La regularización L1 permite identificar las variables más importantes al eliminar aquellas con menor contribución, mientras que L2 mejora la estabilidad del modelo. La regularización no solo mejora el rendimiento del modelo, sino que también aporta interpretabilidad al permitir identificar las variables más influyentes en la clasificación.

```
# Escalar datos
scaler = StandardScaler()
X_train_scaled = scaler.fit_transform(X_train)
X_test_scaled = scaler.transform(X_test)

# L2 (Ridge)
model_l2 = LogisticRegression(l1_ratio=0, C=1.0, max_iter=2000)
model_l2.fit(X_train_scaled, y_train)

# L1 (Lasso)
model_l1 = LogisticRegression(l1_ratio=1, solver='saga', C=1.0)
model_l1.fit(X_train_scaled, y_train)
```



Decision trees

El modelo ha mostrado un buen rendimiento, alcanzando una alta precisión (accuracy), lo que indica que las variables disponibles son adecuadas para la predicción.

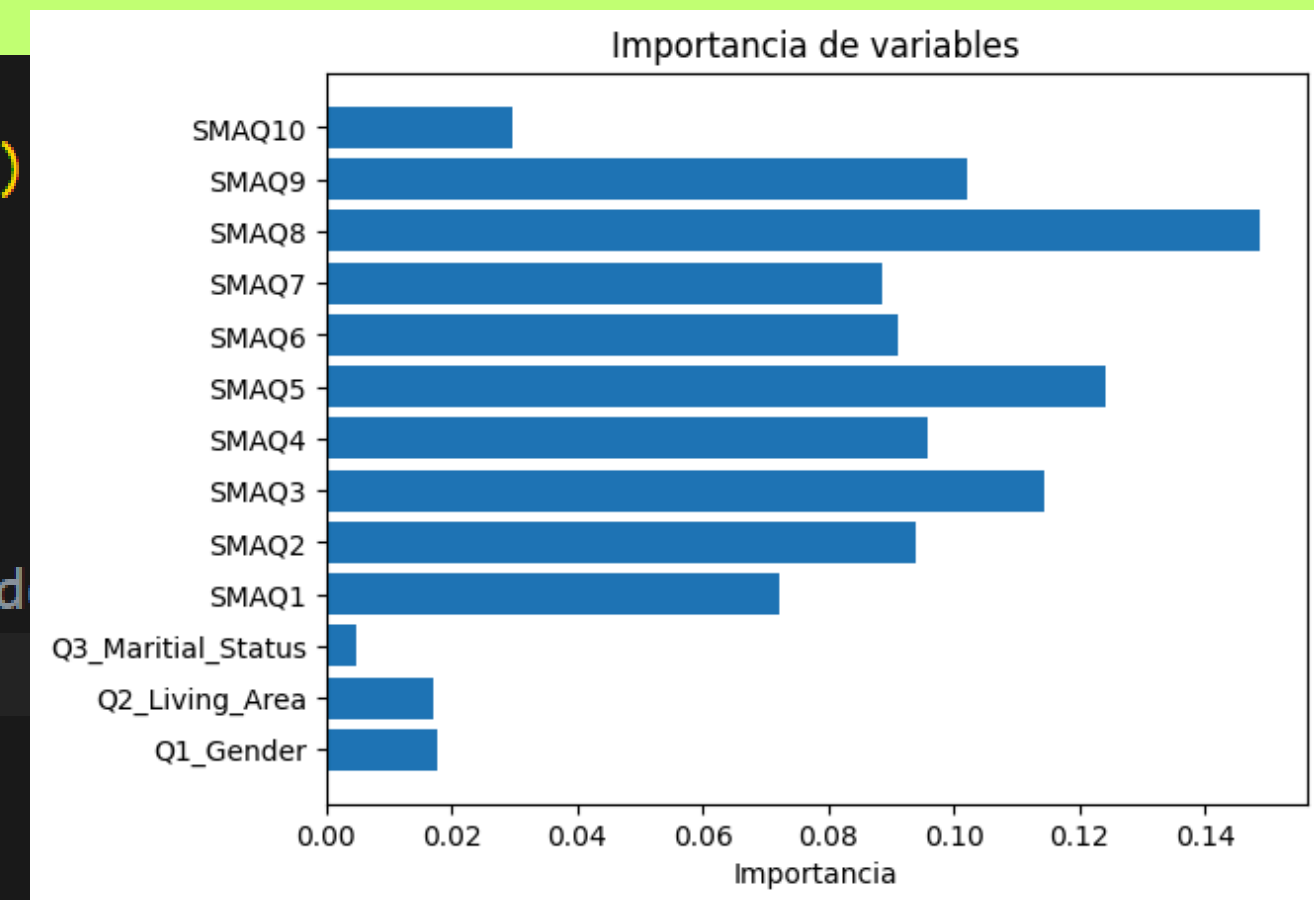
El análisis de importancia de variables revela que los ítems del cuestionario (SMAQ1–SMAQ10) son los principales predictores del nivel de adicción, mientras que las variables demográficas tienen una influencia significativamente menor.

```
# Variables
X = df.drop(columns=["SMA_Scale (Class_Label)", "SMA_Scale_value"])
y = df["SMA_Scale (Class_Label)"]

# División (IMPORTANTE: reproducible)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42    ## rand

)

# Modelo
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)
```



Accuracy: 0.9271844660194175

Ensembles 1

Se han utilizado modelos ensemble como Gradient Boosting y AdaBoost, ya que permiten mejorar el rendimiento mediante la combinación de múltiples modelos. Estos métodos son especialmente adecuados para datasets con relaciones no lineales y presencia de ruido, como es el caso de los datos analizados.

```
# Definir validación cruzada
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42) ##
|     ##shuffle=True para mezclar los datos, random_State: resultados

# Modelos
rf = RandomForestClassifier(random_state=42)
gb = GradientBoostingClassifier(random_state=42)
ada = AdaBoostClassifier(random_state=42)

# Validación cruzada
rf_scores = cross_val_score(rf, X, y, cv=cv, scoring='accuracy')
gb_scores = cross_val_score(gb, X, y, cv=cv, scoring='accuracy')
ada_scores = cross_val_score(ada, X, y, cv=cv, scoring='accuracy')
```

```
Random Forest: [0.91747573 0.93203883 0.9320388
Media RF: 0.9329528771015866
```

```
Gradient Boosting: [0.94660194 0.92718447 0.961
Media GB: 0.9416670613308075
```

```
AdaBoost: [0.89805825 0.9223301 0.92718447 0.8
Media Ada: 0.9027894861472887
```


Ensembles 2 - XGBoost

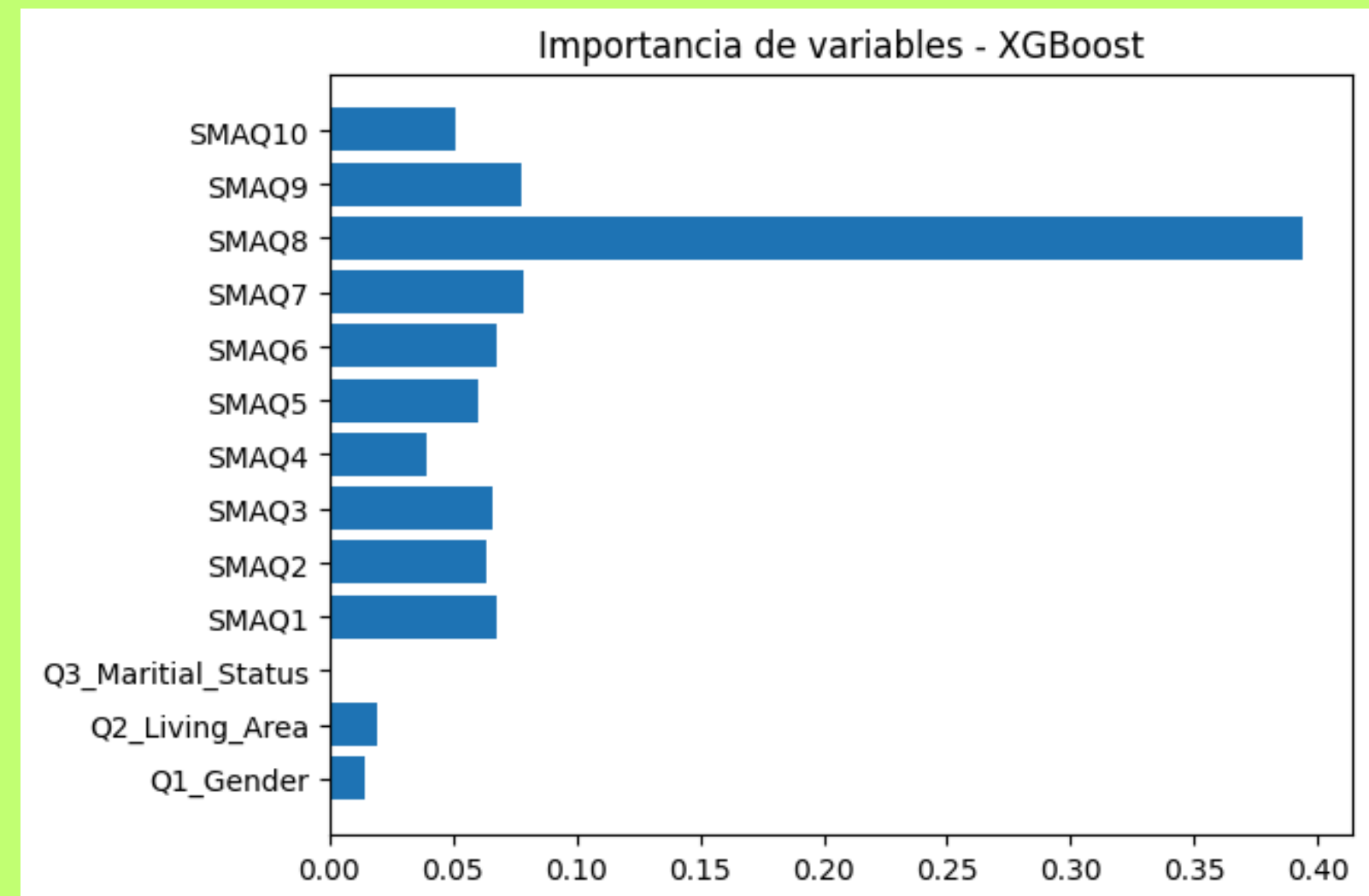
Se ha aplicado el modelo XGBoost, una implementación optimizada de Gradient Boosting, obteniendo un rendimiento competitivo frente a otros modelos ensemble. Su capacidad para manejar relaciones no lineales y su sistema de regularización lo convierten en una de las mejores opciones para este tipo de dataset.

```
# Codificar clases
le = LabelEncoder()          ## transforma clases 0, 1
y = le.fit_transform(y)

# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# Modelo
model = XGBClassifier(
    objective='multi:softmax',
    num_class=len(set(y)),
    eval_metric='mlogloss',
    random_state=42
)

model.fit(X_train, y_train)
```



Accuracy: 0.9466019417475728

Ensembles 3 - LightGBM

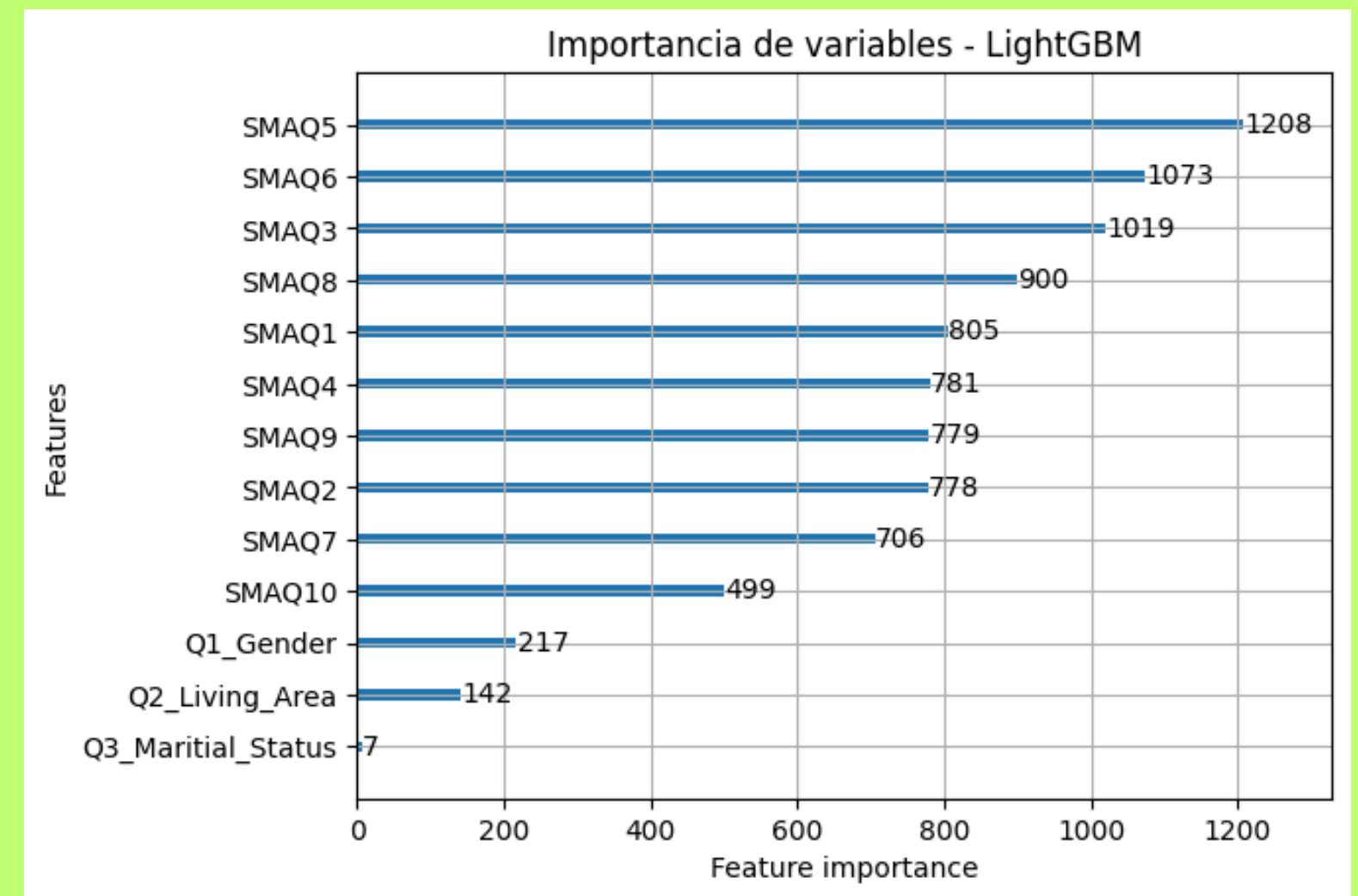
Se ha implementado el modelo LightGBM, un algoritmo de boosting optimizado que permite un entrenamiento eficiente y una alta capacidad predictiva. Los resultados obtenidos son comparables a los de Gradient Boosting y XGBoost, confirmando su idoneidad para datasets tabulares con relaciones no lineales.

```
# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# Modelo
model = lgb.LGBMClassifier(
    objective='multiclass',
    num_class=3,
    random_state=42
)

# Entrenar
model.fit(X_train, y_train)

# Predicción
y_pred = model.predict(X_test)
```



Accuracy: 0.9271844660194175

KNN (K-Nearest Neighbors)

Aplicamos KNN porque es un algoritmo muy útil para clasificación y predicción basada en similitud.

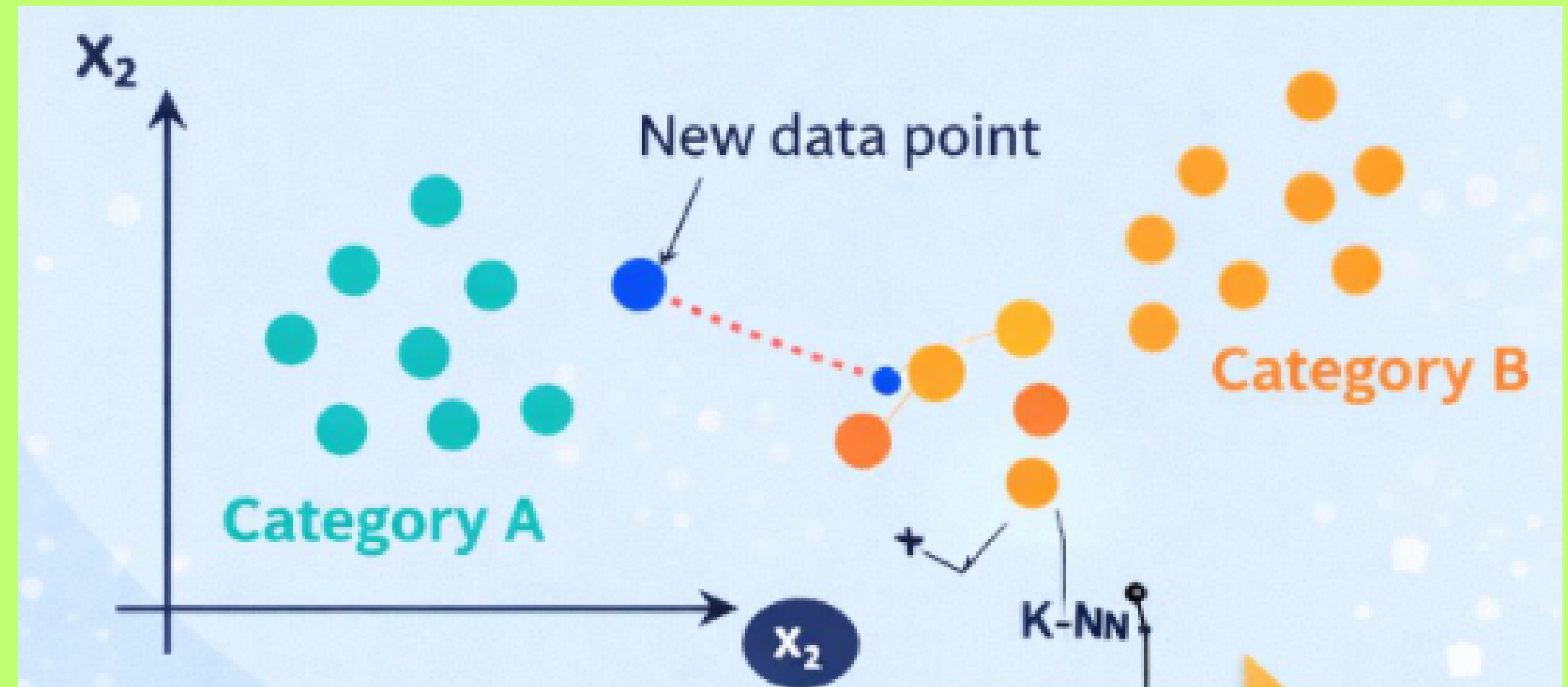
```
# 3. Dividir datos
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)

# 4. Escalado
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 5. Modelo KNN, con K=5
knn = KNeighborsClassifier(n_neighbors=5)

# 6. Entrenar
knn.fit(X_train, y_train)

# 7. Predicción
y_pred = knn.predict(X_test)
```



Accuracy: 0.912621359223301

SVM (Support Vector Machine)

Se compararon dos configuraciones de SVM: kernel lineal y RBF (Radial Basis Function). Los resultados muestran que el **kernel lineal obtiene mejor rendimiento**, aunque existen relaciones no lineales en el dataset.

```
# Split
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.2,
    random_state=42
)
# escalado
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline

svm_rbf = Pipeline([
    ("scaler", StandardScaler()),
    ("svm", SVC(kernel='rbf'))
])

# Validación cruzada
cv = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
```

```
# SVM LINEAL
# =====
svm_linear = SVC(kernel='linear')

scores_linear = cross_val_score(
    svm_linear, X, y,
    cv=cv,
    scoring='accuracy'
)
```

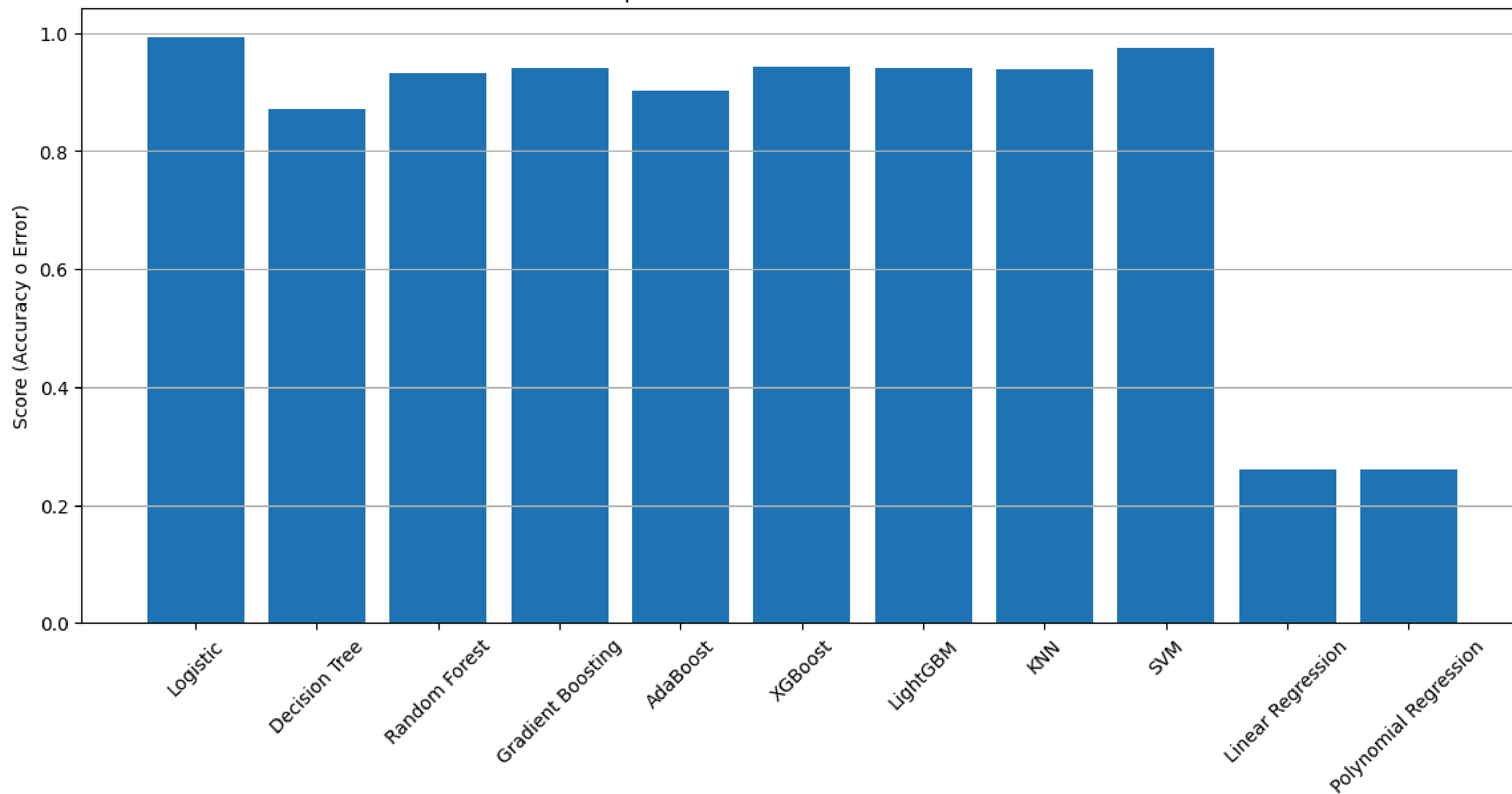
```
# SVM RBF
# =====
svm_rbf = SVC(kernel='rbf')

scores_rbf = cross_val_score(
    svm_rbf, X, y,
    cv=cv,
    scoring='accuracy'
)
```

SVM Lineal: 0.99319914

SVM RBF: 0.97472412976

Comparación FINAL de todos los modelos



Predicción con LightGBM

```
## nuevo caso de SMA
```

```
nuevo_caso = pd.DataFrame([{
    "Q1_Gender": 1,
    "Q2_Living_Area": 2,
    "Q3_Marital_Status": 1,
    "SMAQ1": 3,
    "SMAQ2": 4,
    "SMAQ3": 2,
    "SMAQ4": 5,
    "SMAQ5": 3,
    "SMAQ6": 5,
    "SMAQ7": 5,
    "SMAQ8": 5,
    "SMAQ9": 4,
    "SMAQ10": 5
}])
```

```
1 pred = model.predict(nuevo_caso)
2
3 # Volver a etiquetas originales
4 pred_clase = pred[0] + 1
5
6 print("Clase predicha:", pred_clase)
```

✓ 0.0s

Clase predicha: 3

```
1 proba = model.predict_proba(nuevo_caso)
2
3 print("Probabilidades:", proba)          ## tanto por uno de
```

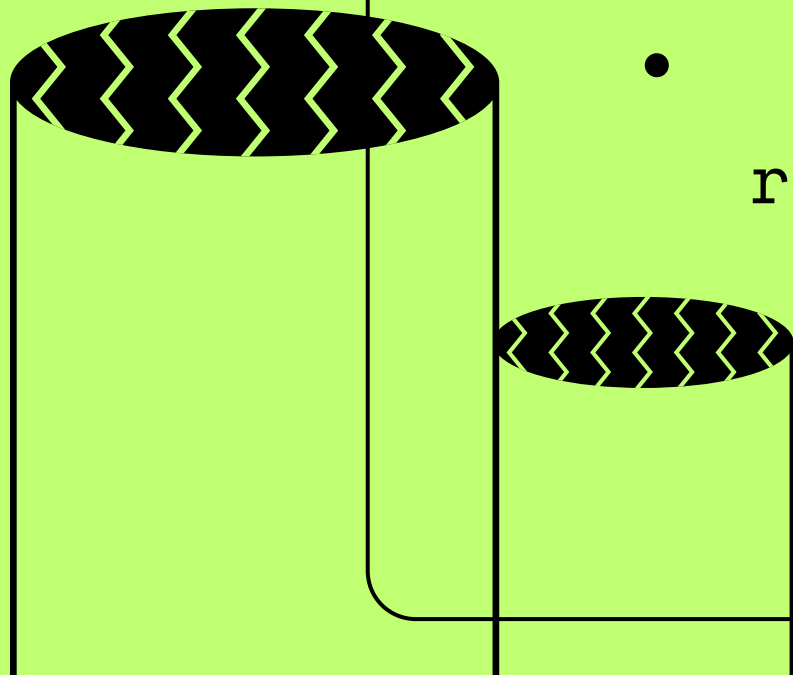
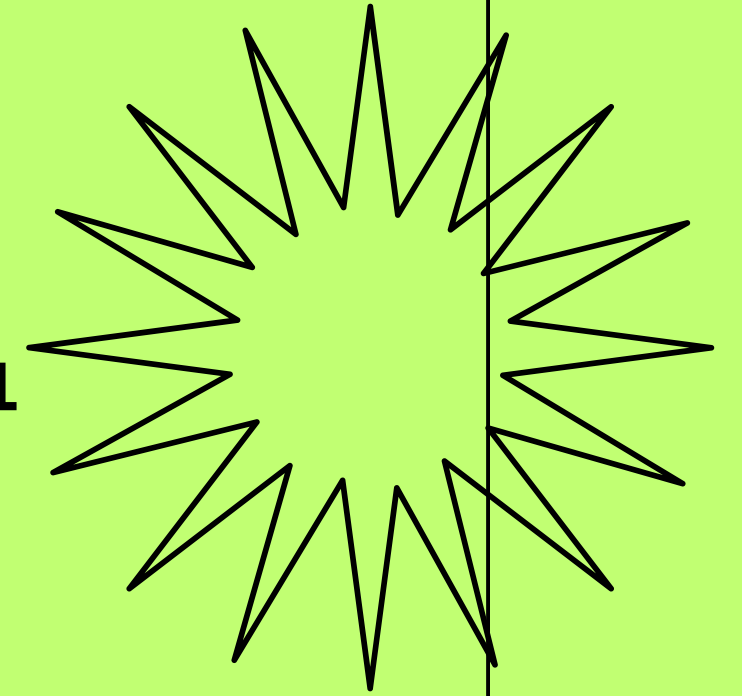
✓ 0.0s

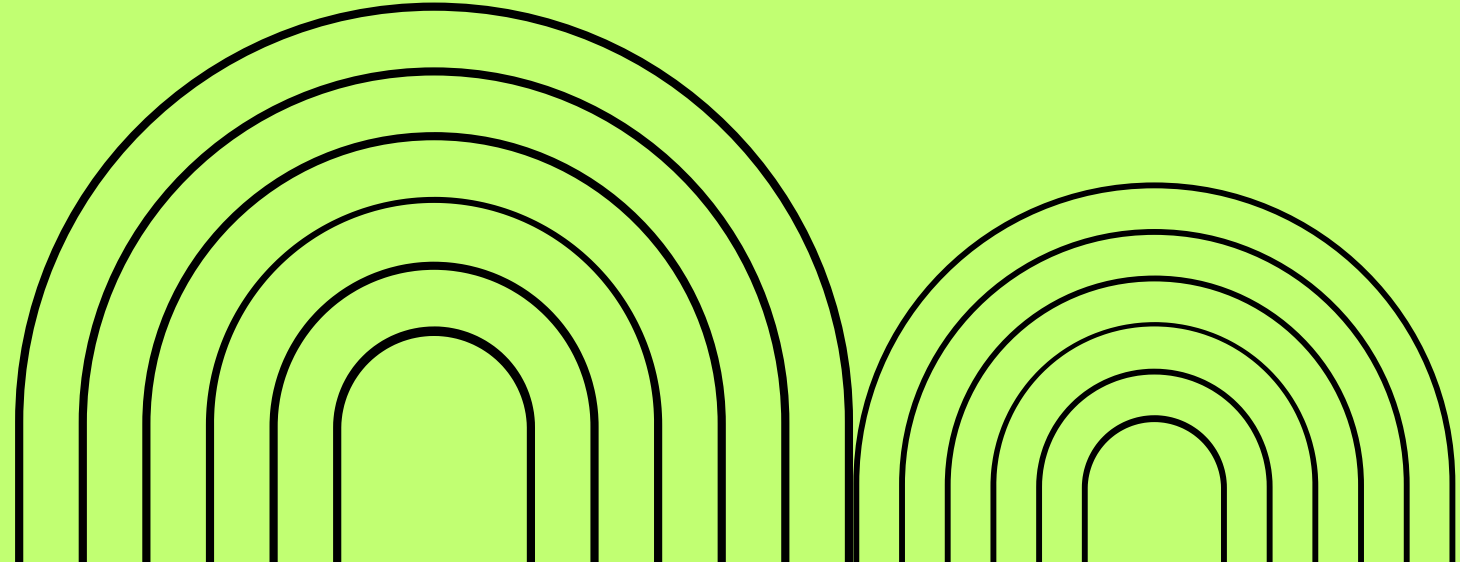
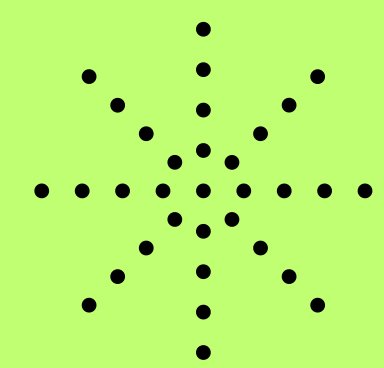
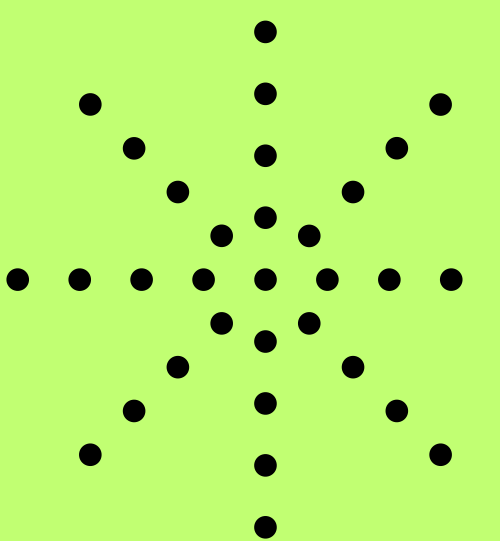
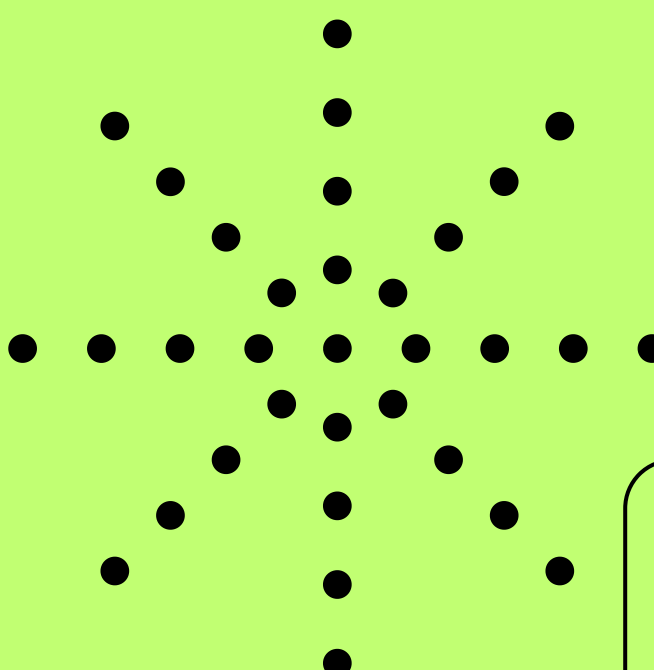
Probabilidades: [[2.04957473e-04 2.48636840e-01 7.51158202e-01]]

Conclusiones

Los métodos de Machine Learning estudiados para el dataset SMA-10 tienen similar potencia, excepto los métodos lineales.

- Logistic Regression funciona peor porque asume relaciones lineales
- SVM puede mejorar pero depende del kernel
- Decision Tree tiende a overfitting
- KNN depende mucho de la escala de los datos
- Random Forest ofrece un mejor **equilibrio** entre rendimiento y robustez, lo que lo convierte en una opción más fiable en escenarios reales.





GRACIAS